

Course Code: SL-3003

Course : Web Engineering Lab

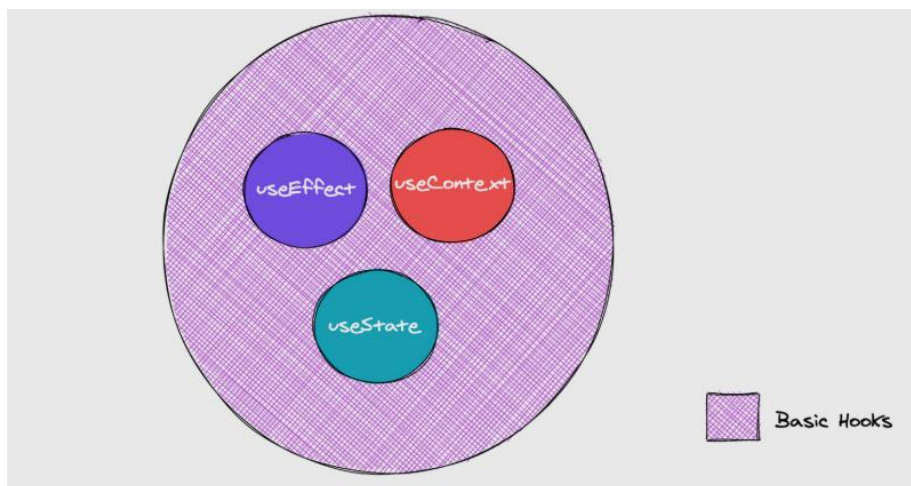
Spring 2025, Lab Manual – 06

React JS

Topic to be studied

React hooks were added to React in version 16.8. With the transition from class to functional components, hooks let you use state and other features within functional components i.e without writing a class component.

This reference guide will discuss all the hooks natively available in React, but first, let's start with the basic React hooks: useState, useEffect



Source code

```
• import React, {useState} from "react";
• function StateFunCom(){
•   // let [userName,setNewName]=useState("asad");
•   let [count,setCounter]=useState(0); //for task
•
•   function FunUpdate(){
•     // setNewName("yasir");
•     setCounter(count+1);
•
•   }
•   console.log("rendering");
•   return(
•     <div>
•       <h1>{count}</h1>
```

```

•         <button onClick={FunUpdate}>click me</button>
•       </div>
•     )
•   }
•   export default StateFunCom;

```

useEffect

Effect Hooks like `useEffect` handle side effects like fetching data, subscriptions, and DOM manipulation.

A side effect is an operation that effect something outside the function scope. In react, side effects are manage using hooks like `useEffect` to ensure they are in a controlled and predictable manner.

`useEffect` is a React Hook that lets you synchronize a component with an external system.

Source code

```

import React, { useEffect } from 'react'
import { useState } from 'react'

const ReactUseEffect = () => {
  const [date, setDate] = React.useState(0);

  useEffect(()=>{
    setInterval(()=>{
      const updatedDate = new Date();
      setDate(updatedDate.toLocaleTimeString());
    },1000)

  },[])

  return (
    <div className='flex justify-center items-center flex-col h-screen'>
      <div className='py-5 bg-blue-600 px-5 border-r-4 '>
        <h1 className='text-white font-semibold '>Date: {date}</h1>
      </div>

    </div>
  )
}

export default ReactUseEffect;

```

Routers in react

React Router is a library for handling routing and navigation in React JS Applications. It allows you to create dynamic routes providing a seamless user experience by mapping various URLs to components. It enables navigation in single-page application (SPA) without refreshing the entire page.

Installation

`npm i react-router`



Link component

In the React application, the Link component is used to navigate between pages without a full-page reload. It replaces the traditional `<a>` tag to improve performance. It is a simple navigator.

Link

```
<Link to="/" className="text-white text-xl hover:text-blue-500 transition duration-300 ease-in-out capitalize font-bold cursor-pointer">Home</Link>
```

NavLink

```
<NavLink to="/profile" className="text-white text-xl hover:text-blue-500 transition duration-300 ease-in-out capitalize font-bold cursor-pointer">Profile</NavLink>
```

Navigation programmatically

Use `useNavigate()` hooks

```
<button onClick={() => navigate('/project')} className="hover:text-gray-300 bg-blue-600 px-4 py-1 rounded-md hover:bg-blue-700 capitalize font-bold cursor-pointer">View Projects</button>
```

Nested routing

```
<Route path="/services" element={<Services/>}>  
<Route path="development" element={<Development/>}/>  
<Route path="hosting" element={<Hosting/>}/>
```

```
</Route>
```

← → ↻ ⓘ localhost:5173/services/development

Dynamic routing

useParams

```
import React from 'react'
import Header from './Header'
import { useParams } from 'react-router'
// import { Link ,Outlet} from 'react-router'
const DevelopmentDetail = () => {
  const { developmentDetail } = useParams();
  return (
    <>
      <Header/>
      <div className='mt-20'>
        <h1 className='text-center py-2 text-amber-800 capitalize'>Detail page
        {developmentDetail}</h1>

      </div>
    </>
  )
}

export default DevelopmentDetail
```

form and inputs validation

Register

Name

Email

Password

Form submitted successfully!

Submit

```
"use client";
import { useState, useEffect } from "react";
import { useFormState, useFormStatus } from "react-dom";
import Header from "../Header";

const validateForm = async (prevState, formData) => {
  const name = formData.get("name")?.trim();
  const email = formData.get("email")?.trim();
  const password = formData.get("password")?.trim();

  const errors = {};

  if (!name) errors.name = "Name is required!";
  else if (name.length < 3) errors.name = "Name must be at least 3
characters!";

  if (!email) errors.email = "Email is required!";
  else if (!/^([^\s@]+@[^\s@]+\.[^\s@]+)$/i.test(email)) errors.email = "Invalid
email format!";

  if (!password) errors.password = "Password is required!";
  else if (password.length < 6) errors.password = "Password must be at least 6
characters!";

  if (Object.keys(errors).length > 0) {
    return errors;
  }
}
```

```

    return { success: "Form submitted successfully!" };
  };

const SubmitButton = ({ isValid }) => {
  const { pending } = useFormStatus();
  return (
    <button
      type="submit"
      disabled={pending || !isValid}
      className={`w-full py-2 rounded-md text-white ${
        pending || !isValid ? "bg-gray-400 cursor-not-allowed" : "bg-blue-500"
      }`}
    >
      {pending ? "Submitting..." : "Submit"}
    </button>
  );
};

const Register = () => {
  const [state, formAction] = useFormState(validateForm, {});
  const [formData, setFormData] = useState({ name: "", email: "", password: "" });
  const [errors, setErrors] = useState({});

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData({ ...formData, [name]: value });

    let error = "";
    if (name === "name") {
      if (!value.trim()) error = "Name is required!";
      else if (value.length < 3) error = "Name must be at least 3
characters!";
    }
    if (name === "email") {
      if (!value.trim()) error = "Email is required!";
      else if (!/^[\s@]+\.[\s@]+\.[\s@]+$/\.test(value)) error = "Invalid
email format!";
    }
    if (name === "password") {
      if (!value.trim()) error = "Password is required!";
      else if (value.length < 6) error = "Password must be at least 6
characters!";
    }

    setErrors({ ...errors, [name]: error });
  };
};

```

```

};

const isValid = Object.values(formData).every((val) => val.trim() !== "") &&
    Object.values(errors).every((err) => err === "");

useEffect(() => {
    if (state && !state.success) {
        setErrors(state);
    }
}, [state]);

return (
    <>
    <Header/>
    <div className="h-screen flex justify-center items-center">
        <form action={formAction} className="bg-white p-6 rounded-lg shadow-md
w-80">
            <h2 className="text-xl font-bold mb-4">Register</h2>

            <label className="block mb-2">Name</label>
            <input
                type="text"
                name="name"
                value={formData.name}
                onChange={handleChange}
                className="w-full p-2 border rounded-md mb-2"
            />
            {errors.name && <p className="text-red-500">{errors.name}</p>}

            <label className="block mb-2">Email</label>
            <input
                type="email"
                name="email"
                value={formData.email}
                onChange={handleChange}
                className="w-full p-2 border rounded-md mb-2"
            />
            {errors.email && <p className="text-red-500">{errors.email}</p>}

            <label className="block mb-2">Password</label>
            <input
                type="password"
                name="password"
                value={formData.password}
                onChange={handleChange}

```

```

        className="w-full p-2 border rounded-md mb-2"
      />
      {errors.password} && <p className="text-red-500">{errors.password}</p>

      {state?.success} && <p className="text-green-500">{state.success}</p>

      <SubmitButton isValid={isValid} />
    </form>
  </div>
</>
);
};

export default Register;

```

task1

Dynamic navbar

Create a dynamic navbar in JavaScript using `useState` for dropdown functionality and implement dynamic routing.

- Categories: Home, Products
- The "Products" category will have subcategories such as "Home Appliances" and "Kitchen Appliances."
- Each subcategory will contain at least 5 products.
- Each product will have 5 sub-products.
- Clicking on a product should open its respective page.
- Clicking on a sub-product should open its respective page.
- The page should be responsive using Tailwind CSS

Task2

Task Manager System

Create a Task Manager System using JavaScript and React. The system should include:

- User Input Box to add tasks.
- Task List (To-Do List) where tasks will be displayed.
- If a task already exists, an error message should appear.
- Each task should have two buttons:
 1. Complete Task – Moves the task to the Completed Task List.

2. Delete Task – Shows a confirmation alert before deleting the task from the list.
- Responsive design using Tailwind CSS

Task3

Detail page

Create a nested routing structure using React Router. The project should include:

- A Navbar with links to Categories and Products.
- The Products page should display a list of products.
- Clicking a product should navigate to a Product Details page using dynamic routing (by passing the product id in the URL).
- The Product Details page should fetch and display the details of the selected product.
- The project must be fully responsive using Tailwind CSS.

Task4

cart system

Implement a shopping cart system using React hooks where:

- Users can add products to the cart.
- A cart counter is displayed in the basket.
- The cart data is displayed in a structured Tailwind CSS table.
- Validation ensures that no product quantity exceeds 5.
- The counter cannot go below 0.

Task 5

dashboard

Create a static dashboard where two types of users can log in. If a user logs in, they should be redirected to the user dashboard, and if an admin logs in, they should be redirected to the admin dashboard. The values can be static, but routing should be dynamic. The user type (admin or user) will be stored and used for routing, ensuring correct redirection.

The form should have validation, checking both input fields and the submit action. The design should be fully responsive using Tailwind CSS.

Task6

Ticker

Create a ticker at the bottom of the page that updates in real-time at a set interval, displaying the current date and time. The ticker should not be visible on small screens.

